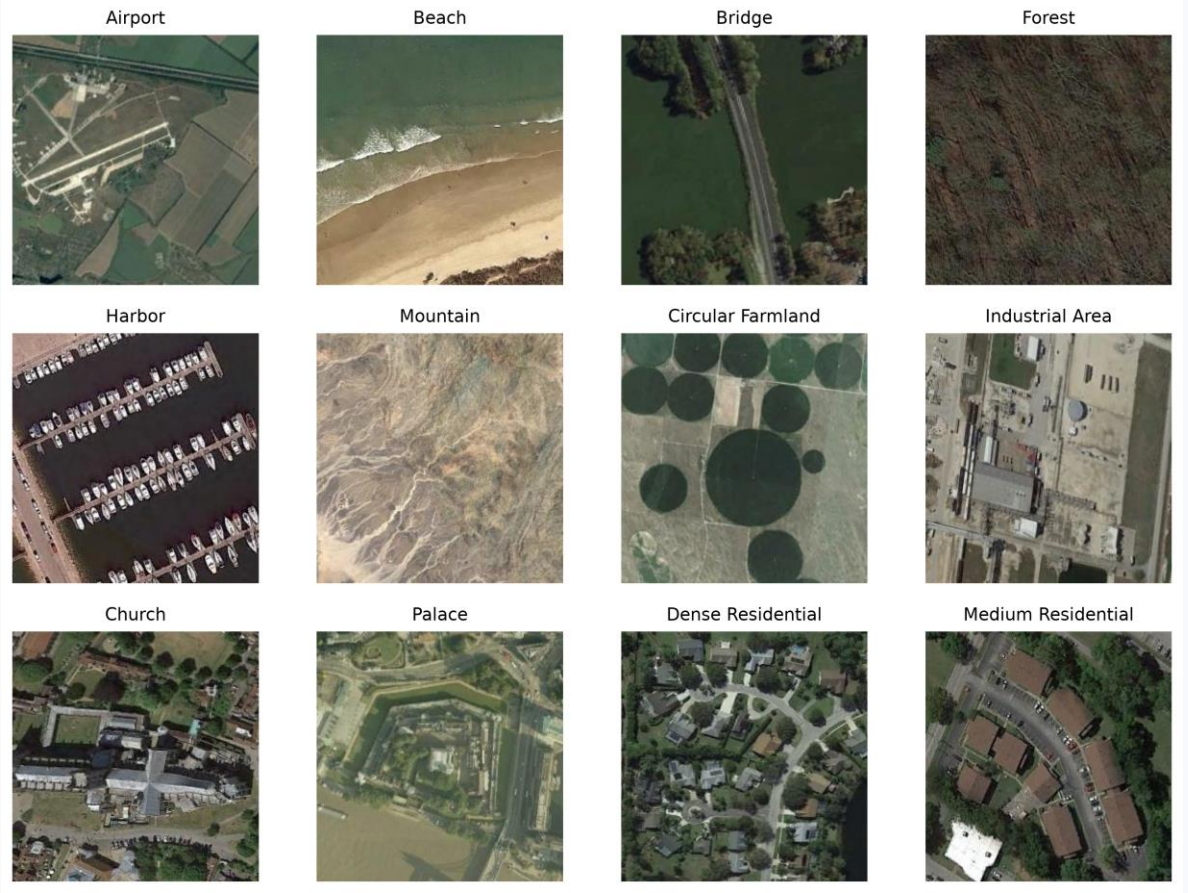
Authors: Shubh Gupta, Luka Waronig

SEEING EARTH AT THE EDGE



Adaptive inference vision transformers for remote sensing scene classification in resource-constrained deployments



54%
FLOPs reduction (computation saved)

94%
(only ~1% drop)
Adaptive inference mode accuracy

2 ms
(~3x speed-up)
Median CPU latency of one inference

NWPU-RESISC45 • DeiT-Tiny Vision Transformer • adaptive execution depth by confidence-based early exits at blocks 4 / 8 / 12

Problem & research question

Can a vision transformer spend computation only on the hard satellite images?

EDGE CONSTRAINTS

- Compute and latency budgets fluctuate
- Full-depth inference wastes work on easy inputs
- A static shallow model hurts difficult examples

RESEARCH QUESTION

Can a vision transformer model allocate computation according to each input image’s difficulty?

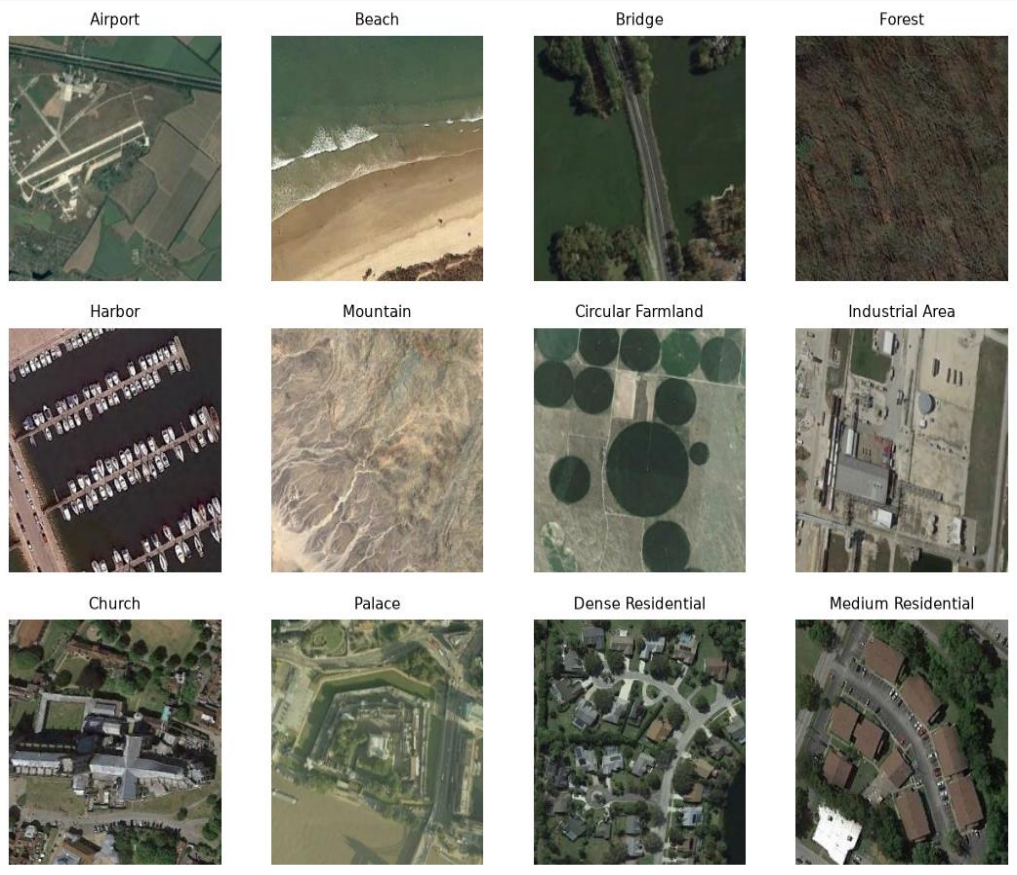
Conditional computation

Adaptive transformer depth

Accuracy–FLOPs frontier

Dataset & evaluation split

NWPU-RESISC45: aerial 256x256 images balanced 45-class remote-sensing scene classification benchmark



31,500

RGB images

45

Balanced classes

Published splits and policy isolation

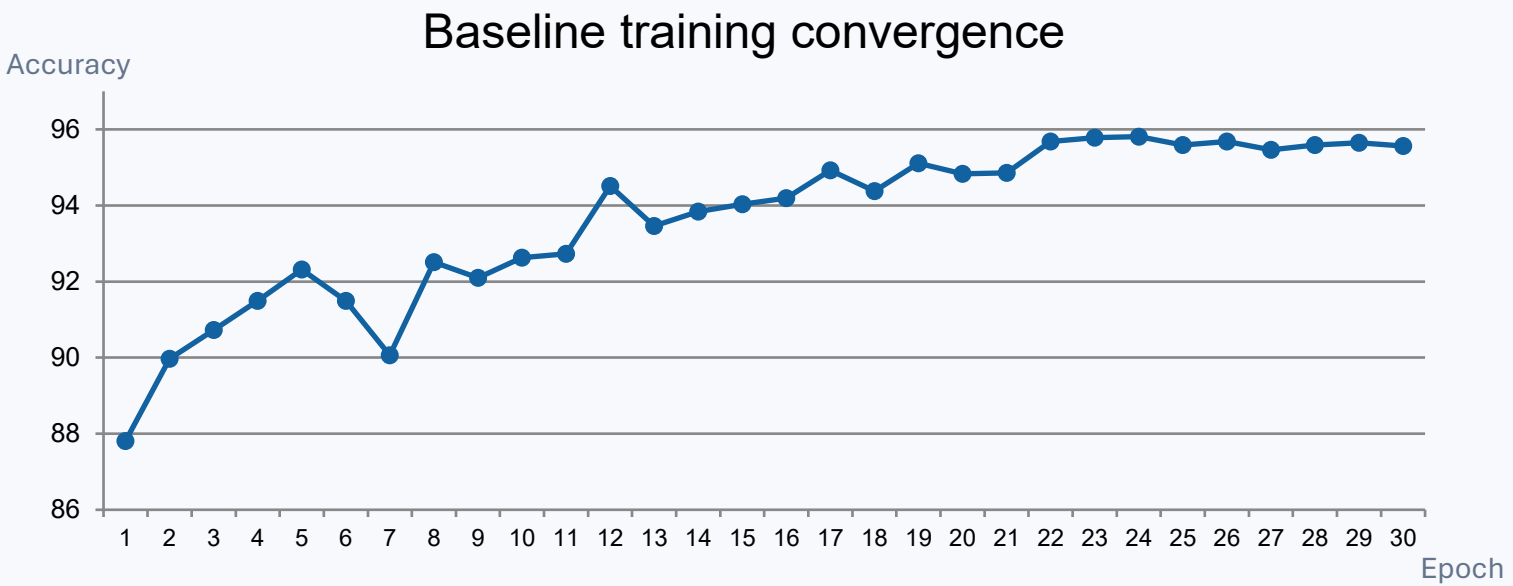
Partition	Images	Purpose
Train	18,900	Fit baseline + adaptive student
val_select	3,150	Checkpoint selection
val_policy	3,150	Reserved for calibration + policy selection
Test	6,300	Final held-out evaluation

val_policy is split again half-half into calibration and policy-selection images. Test data is obviously not used to choose thresholds.

Step 1: Freezing a strong fine-tuned static reference

ImageNet pre-trained; **DeiT-Tiny vision transformer** processes every input through all **12 transformer blocks**

5.53 M	1.274	95.81%	94.71%
Parameters	GFLOPs / image	Validation accuracy	Test accuracy

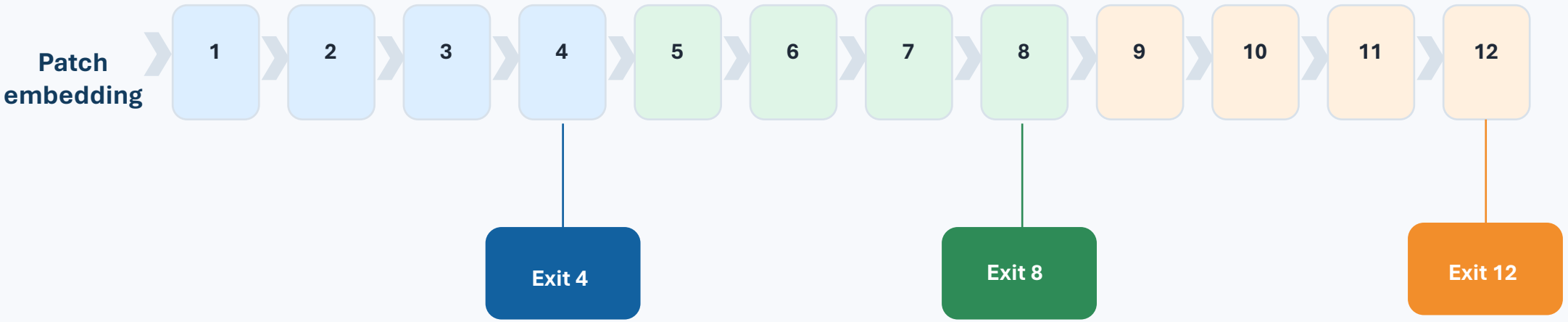


Examples of difficult classes:

Palace - 79.10%
Church - 84.38%
Medium residential - 85.29%
Dense residential - 87.18%
Industrial area - 90.00%

Step 2: Shared early-exit transformer

One backbone, two lightweight intermediate heads within the same network



Confidence ≥ threshold? → return prediction
Otherwise continue only if the next depth fits the budget.

Selected policy on held-out test:

- 67.75% exit at block 4
- 30.65% exit at block 8
- 1.60% reach block 12

Implementation: early-return prediction path

Excerpt from the implementation notebook: model stops during execution, not after computing all 12 blocks

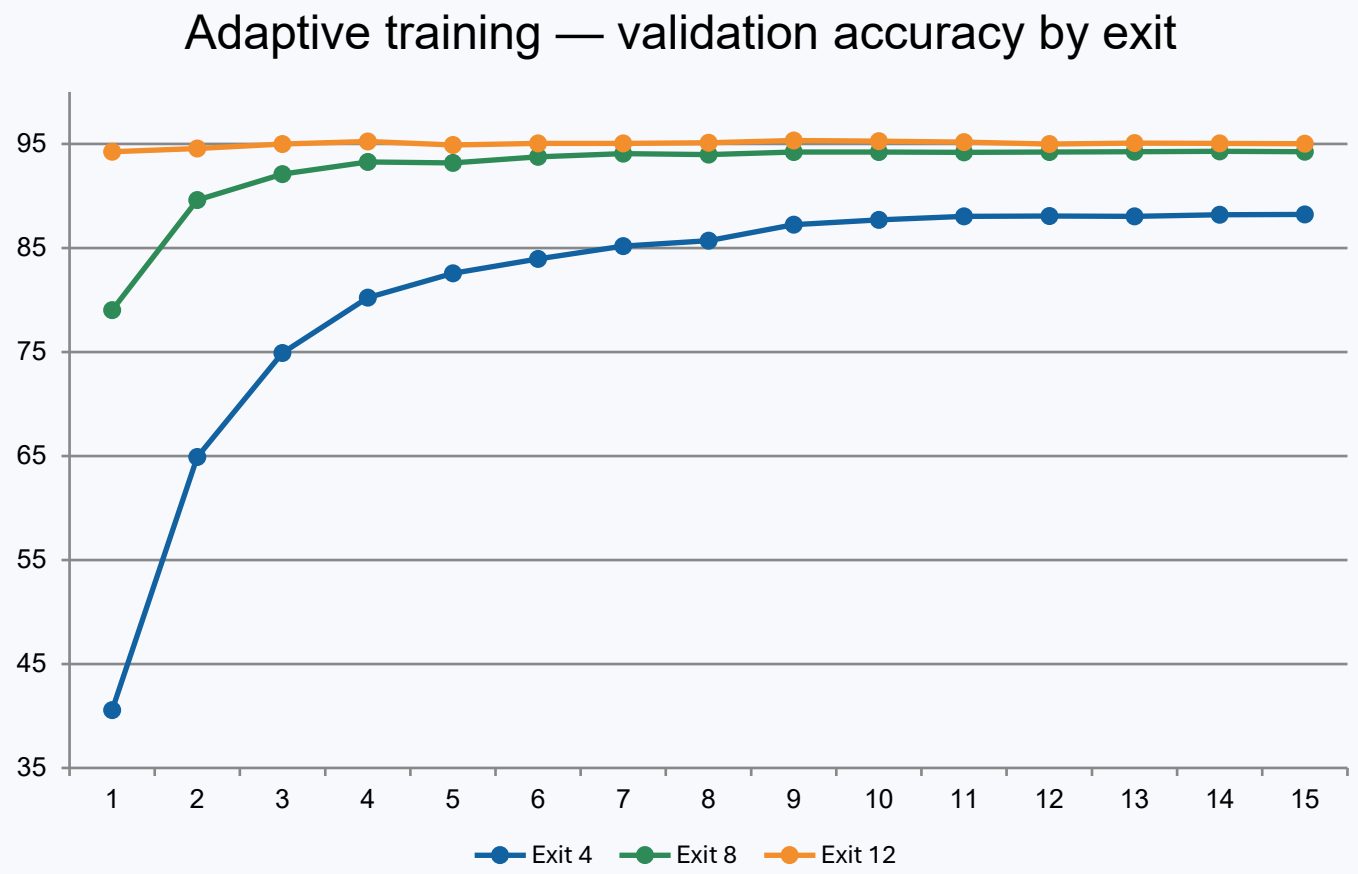
```
@torch.inference_mode()
def infer_one(self, image, thresholds, temperatures, max_depth=12):
    x = self.embed(image)
    for depth, block in enumerate(self.backbone.blocks, 1):
        x = block(x)
        if depth in EXITS:
            logits = self.classify(x, depth)
            confidence = (logits / temperatures[depth]) \
                         .softmax(-1).amax(-1).item()
            if depth == max_depth or depth == 12:
                return logits, depth, "budget" if depth < 12 else "final"
            if confidence >= thresholds[depth]:
                return logits, depth, "confidence"
```

What this guarantees?

- Blocks after the selected exit are never run
- Confidence cost is included in path profiling
- A resource cap can force an earlier return
- Direct path is checked against cached decisions on 8 samples

Training the early exits with knowledge distillation

Ground-truth supervision + frozen full-depth teacher



How the early exits learn?

Ground-truth labels
Correct scene class

Frozen teacher
Soft class probabilities

Early-exits (blocks 4, 8)
Learn from both signals

88.03%	94.19%	95.17%
Exit 4 val	Exit 8 val	Exit 12 val

Selected best checkpoint: epoch 11

Calibration & stopping-policy selection

Two-stage val_policy split keeps calibration separate from threshold selection

1) Temperature calibration

Temperatures are fit by minimizing cross-entropy on the calibration subset only.

Test-set calibration diagnostic: ECE

2) Policy selection

Frozen teacher accuracy: 95.96%
Allowed tolerance: −1.00 pp → target ≥ 94.96%

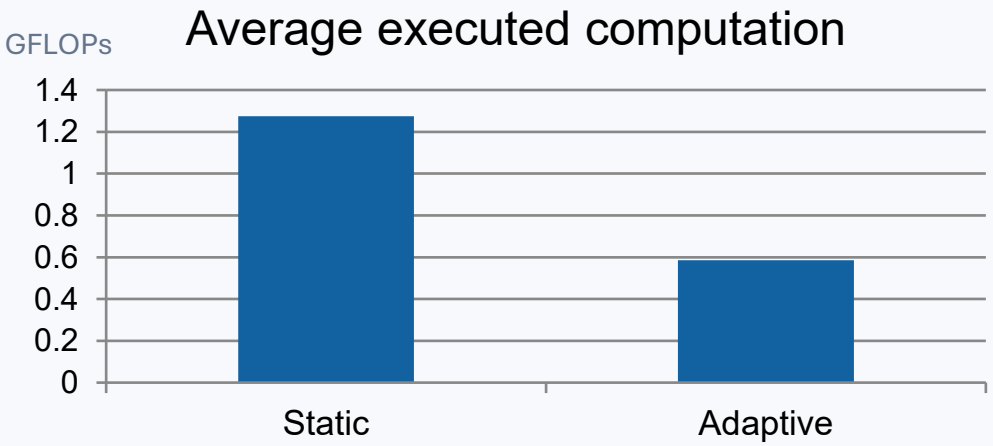
Selected thresholds

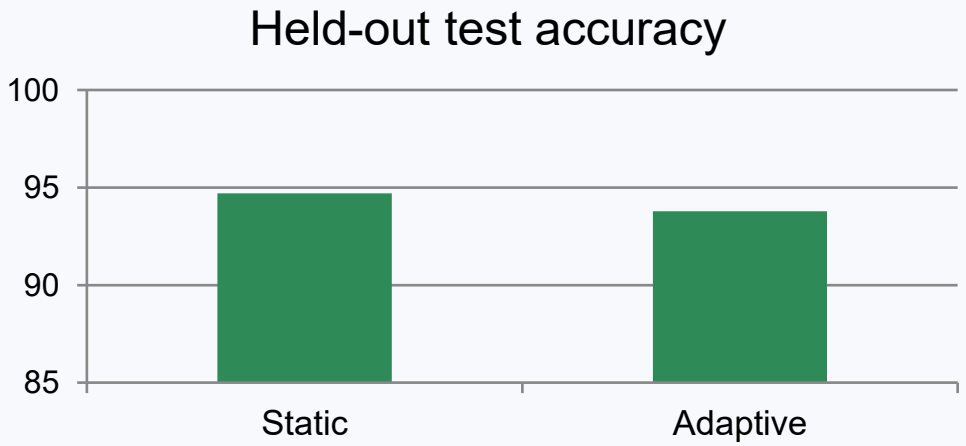


Main performance result: half the computation, nearly all accuracy

Held-out test set: 6,300 images

94.71%	93.79%	−0.92 pp	54.13%
Static accuracy	Adaptive accuracy	Accuracy change	FLOPs saved

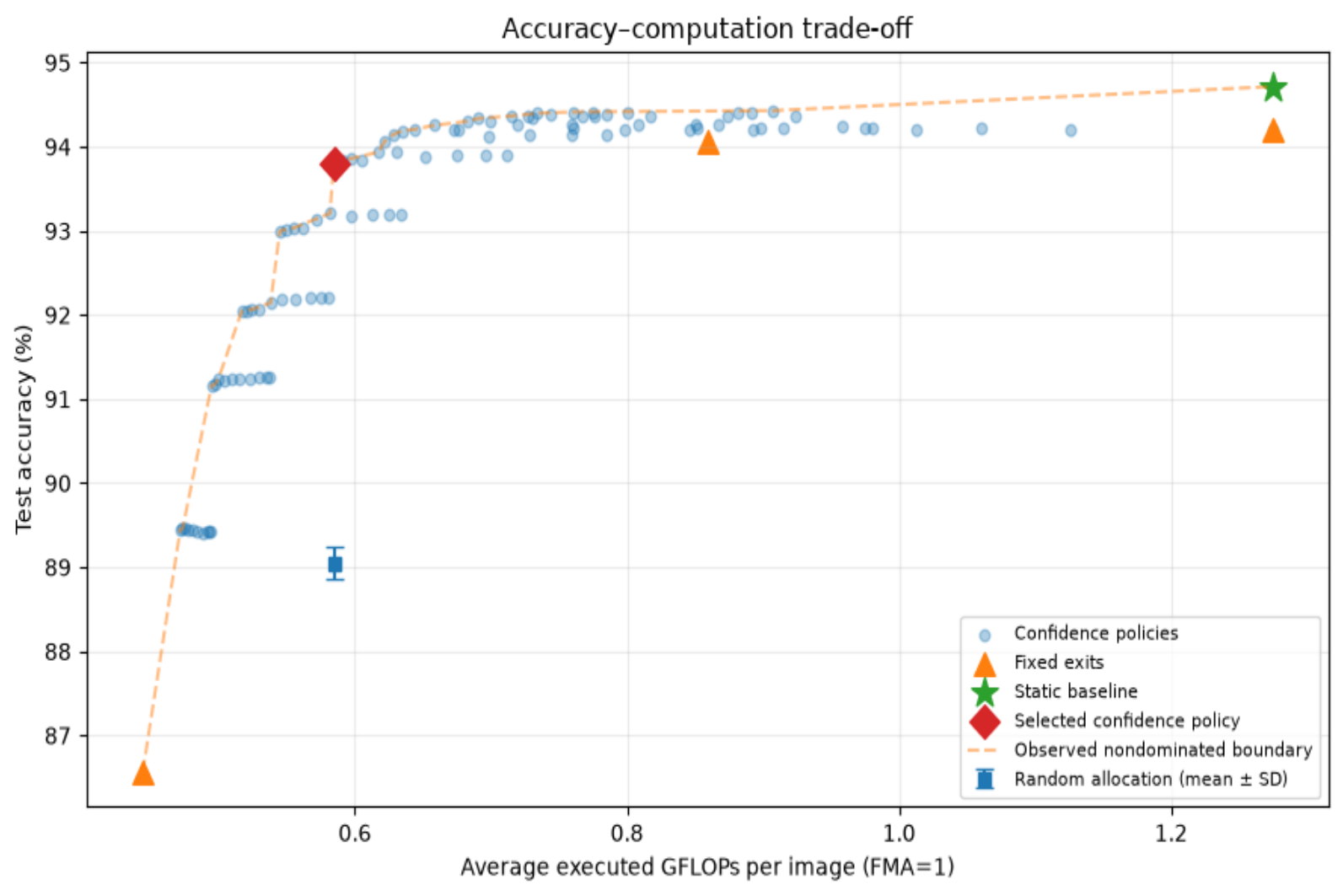




The adaptive system executes only 45.9% of the baseline FLOPs on average.

Accuracy–computation frontier

Confidence-guided execution creates useful operating points between fixed depths



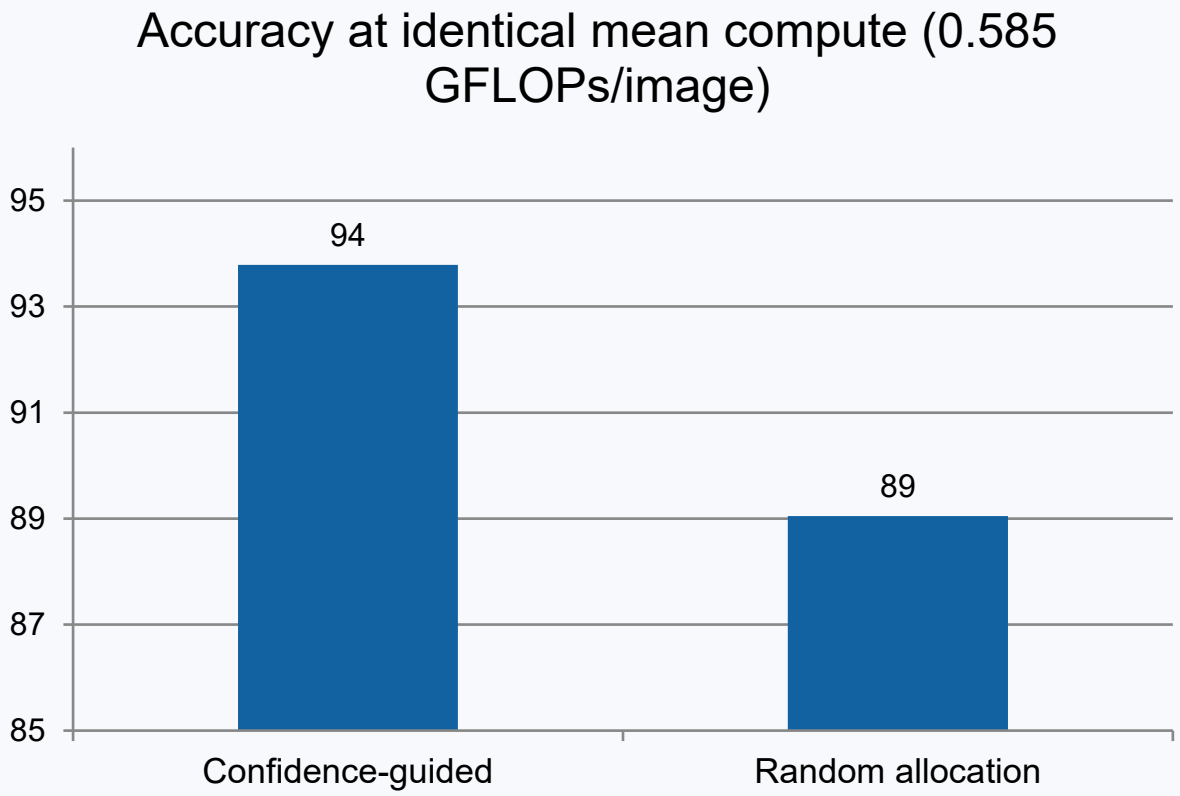
Reference points

Mode	Accuracy	GFLOPs
Fixed 4	86.56	0.444
Adaptive	93.79	0.585
Fixed 8	94.06	0.859
Fixed 12	94.21	1.274
Static	94.71	1.274

Key idea: input-dependent allocation beats a single fixed depth when inputs vary in difficulty.

Does confidence choose the right images?

Control experiment: same exit counts, same mean FLOPs, random image assignment



+4.74 pp

Confidence advantage

±0.19

Random SD (30 runs)

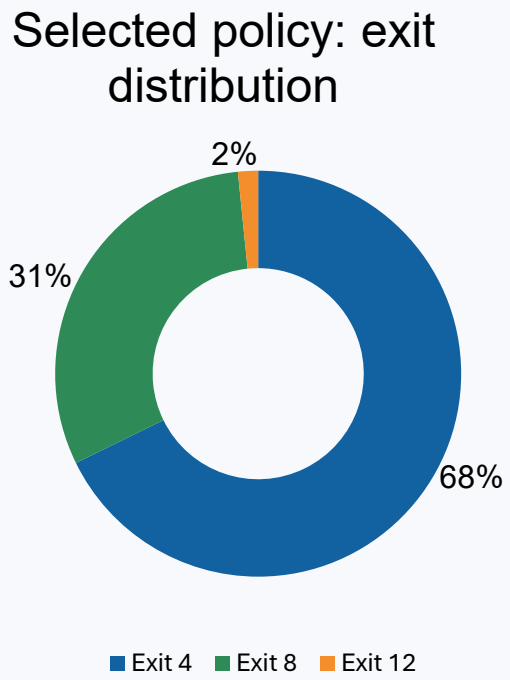
Deeper processing changes individual predictions

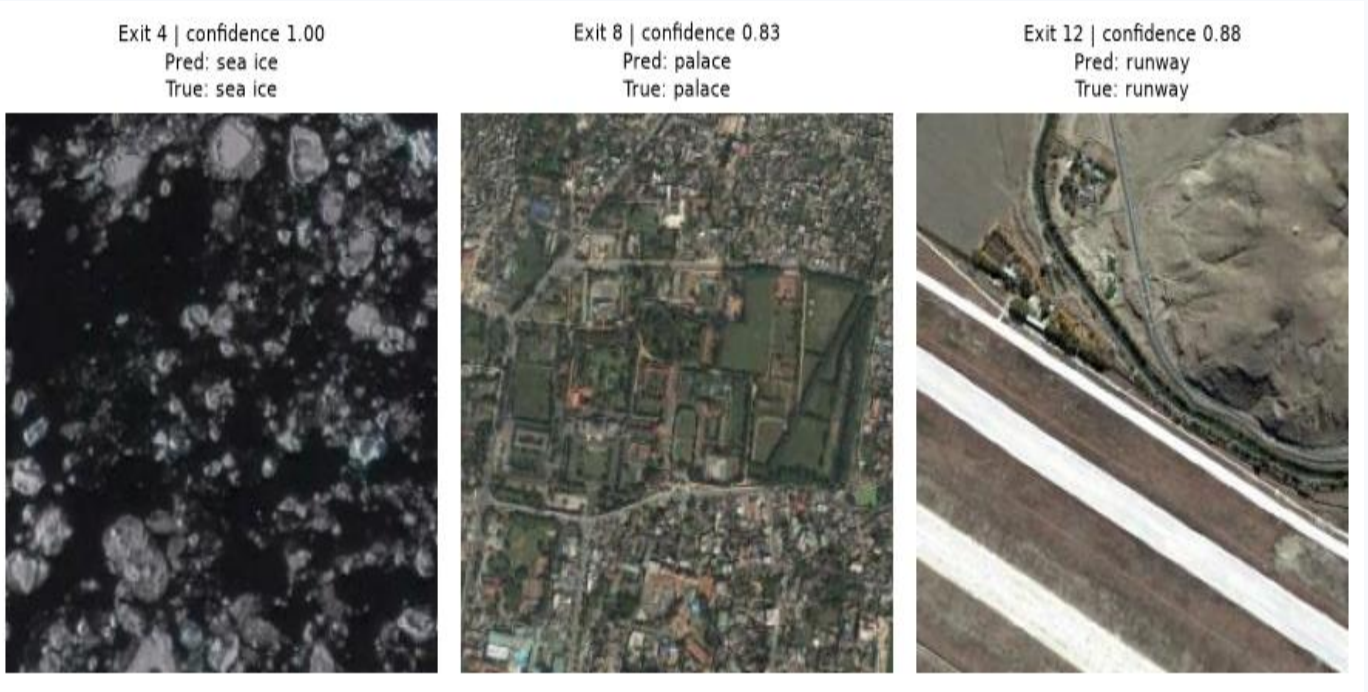
Transition	Errors corrected	Correct lost
4 → 8	574	101
8 → 12	118	109

Conclusion: confidence-based input selection is materially better than random path assignment at the same exit counts.

What the adaptive model actually does

Most images are easy enough to stop early

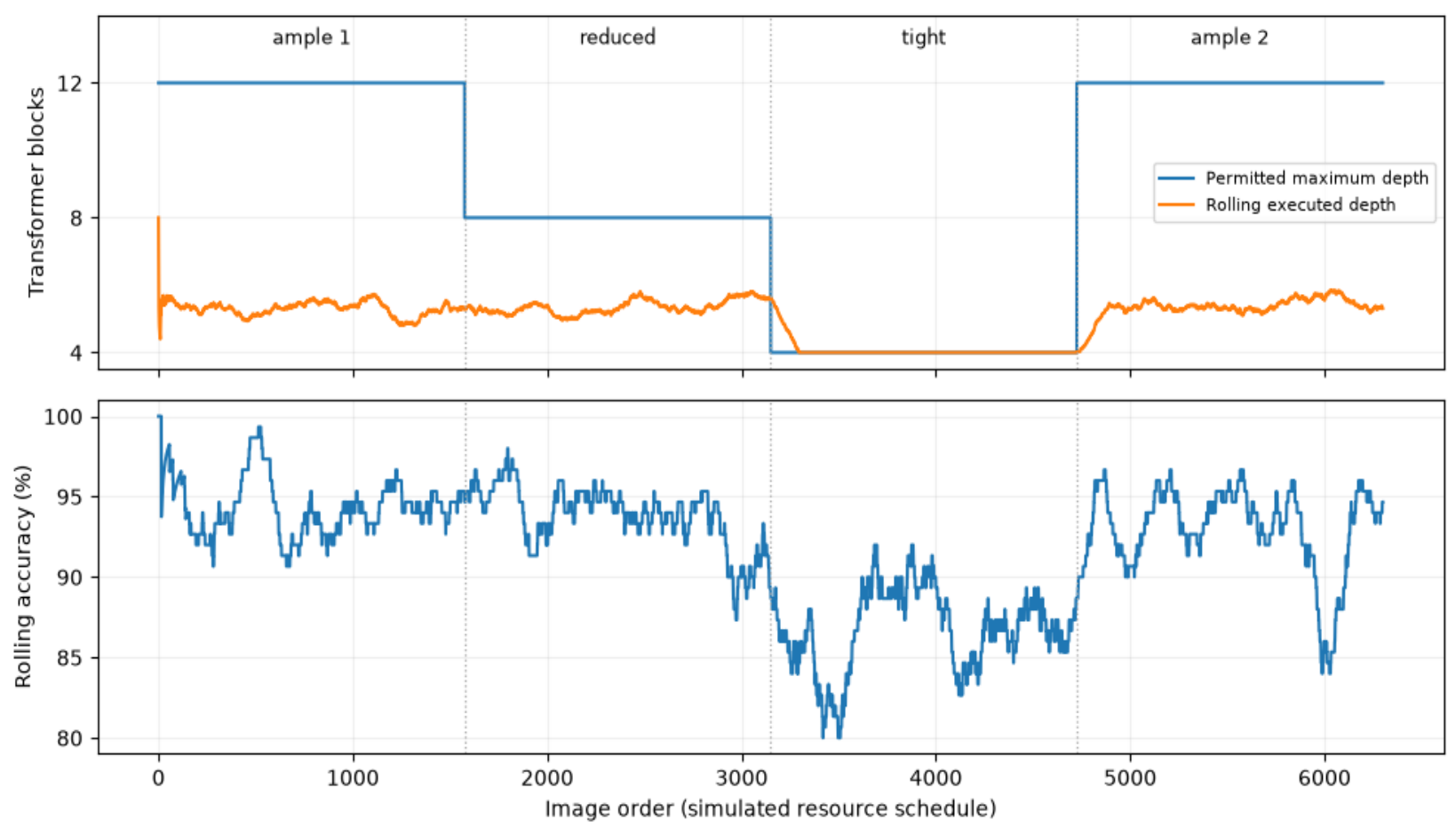




Interpretation: 98.4% of test images avoid the final 12-block route. Only the most uncertain cases consume full depth.

Resource-aware inference

The same controller can obey a defined maximum execution depth



Top: four equal replay phases (1,575 images each). Bottom: each cap evaluated separately on all 6,300 test images.

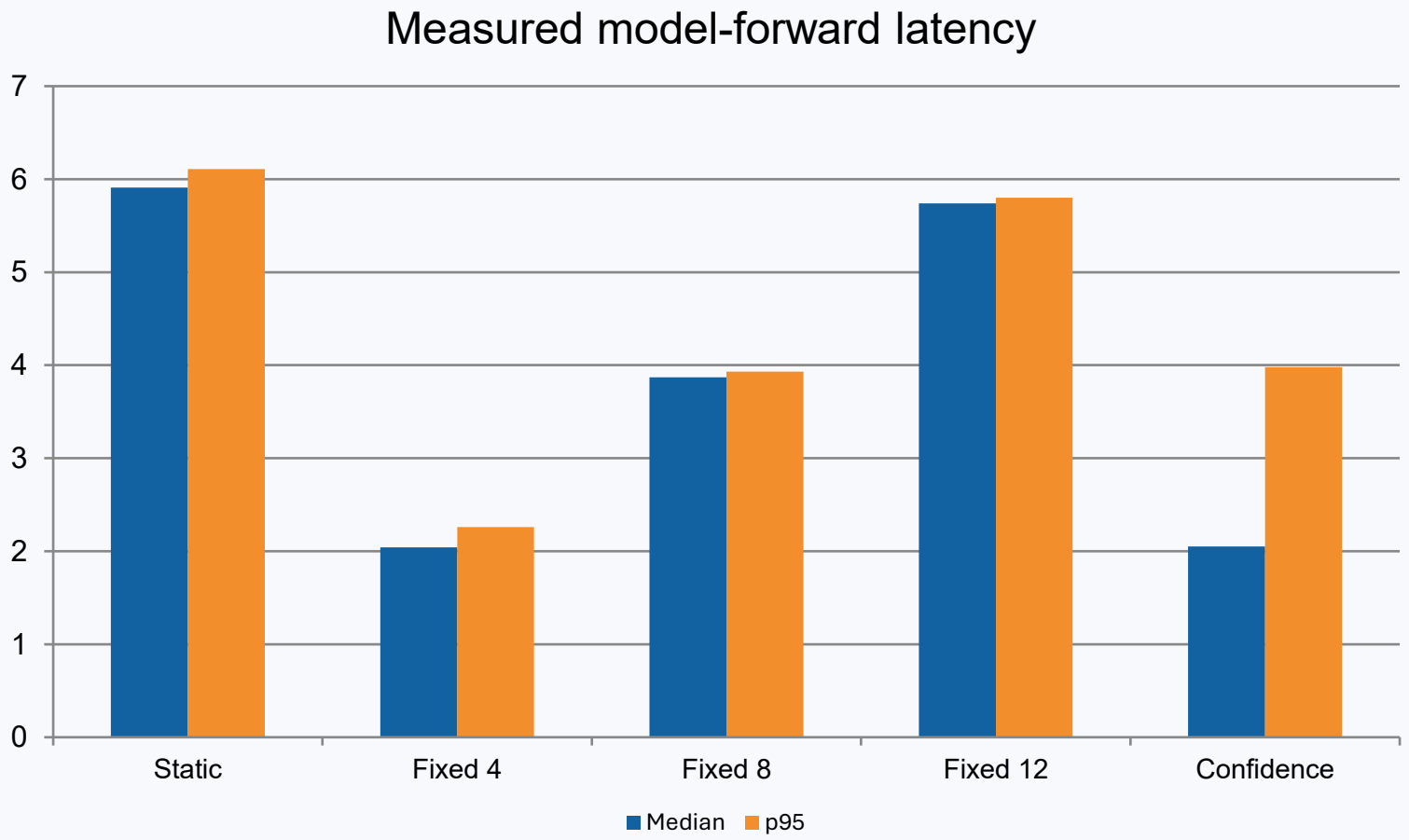
Full test under fixed caps

Max depth	Accuracy	GFLOPs
4 (tight)	86.56%	0.44
8 (moderate)	93.62%	0.58
12 (full)	93.79%	0.58

A moderate cap preserves nearly all accuracy. A tight cap forces earlier predictions, even when the model remains uncertain.

FLOP savings and measured CPU latency savings

Common rebenchmark: 256 sampled test images • batch-one • CPU FP32 • one thread



65.3%

Median latency reduction

34.9%

p95 latency reduction

Static: 5.91 / 6.11 ms
Confidence policy: 2.05 / 3.98 ms
(median / p95; model-forward only)

Takeaways, limitations & next steps

What we learned, and what remains...

Findings

- 54% fewer FLOPs for only −0.92 pp on accuracy (and ~3x faster)
- Confidence beats random allocation at equal compute
- Controller obeys external resource caps

Limitations

- Benchmark image scenes, not a live stream of frames
- Latency measured on CPU, not edge / embedded hardware
- One backbone family / one dataset

Next steps

- Measure energy and latency on edge hardware
- Adapt budgets dynamically from device state
- Test domain drift and calibration robustness

Conclusion: adaptive depth is a practical way for vision transformers to trade a small amount of accuracy for large average compute savings in remote-sensing scene classification settings.

Appendix: Table of key numerical results

Configuration	Accuracy (%)	GFLOPs / image	Median (ms)	Tail p95 (ms)
Static baseline	94.71	1.274	5.91	6.11
Fixed depth 4	86.56	0.444	2.04	2.26
Fixed depth 8	94.06	0.859	3.87	3.93
Fixed depth 12	94.21	1.274	5.74	5.80
Confidence-policy (adaptive)	93.79	0.585	2.05	3.98
Random allocation*	89.05 ± 0.19	0.585	—	—

Note: Fixed-depth modes use the jointly trained adaptive model, while the static baseline is a separately fine-tuned reference. CPU latency is measured on a separate 256-image reference subset.